

Production readiness audit

Vetting questions, and what the code actually says. 2026-08-06.

Verdict up front: the trading loop is closer to production-ready than the learning loop. Risk gating, order handling and kill switches are genuinely defensive and well tested. The memory spine is half-wired — its central premise is that beliefs are earned from outcomes, and the mechanism that would earn them is not connected.

Q1. Does it robustly follow journals and lessons?

Journals: yes. Eleven event kinds are written (take, committee, agent_pm, historian, episode, lesson_*, operator_mandate, dossier_update). Append-only by design — no DELETE exists in `memory/store.py`. Read by the PM, the historian, the copilot and the dashboard.

Lessons: partially, and the gap is structural.

Finding 1 — **add_episode** is never called. The episodes table is defined, indexed, and has a fully-written insert method. Nothing in `src/` calls it. So:

- `lesson_evidence.episode_id` never points at a real closed trade.
- `historian.py:223` passes a synthetic `week_tag` instead.
- Therefore lesson promotion (+3 net support → established) is driven by an LLM voting weekly on its own lesson book, not by outcomes.
- The entry `pctile_52w`, `pnl_pct` and context columns designed to make lessons falsifiable are unused.

The docstring in `memory/store.py` states the rule as "Everything gradeable... Skill is a number attached to memory, not a vibe." For predictions that is true. For lessons it is currently aspirational.

Finding 2 — **cited_lessons** is collected and never read. Every take schema asks agents to cite lesson IDs. No code consumes the field. So there is no measure of which lessons get used, and no way to retire one nobody ever cites. Two lines of plumbing away from being a real signal.

Finding 3 (fixed 2026-08-06). Only established lessons reached the context, so a tentatively-phrased lesson influenced nothing and could never accumulate evidence — nothing trades on a lesson no agent can see. Candidates now arrive as `operator_lessons_under_consideration`.

Q2. Does it track its actions and evolve direction dynamically?

Tracking: strong. Orders, fills, snapshots, equity curve, per-agent predictions with outcomes and Brier scores, source trust as a Beta posterior, and a shadow book of considered-and-passed names with forward returns against benchmark. That last one is better than most retail desks have.

Evolving: this is where the month of drift happened. Documented in full in the git history of `agents/pm.py`. Summary:

- The PM had no candidate feed. Nothing in its prompt named a ticker it did not already hold; the 1,600-name whitelist arrived as a string. Its free-associated, and free association is stable.
- Its charter named JPM and LMT as examples, which became the candidate list.
- Agent takes never reached it — only the manager's synthesis — so the "creative/scout bullish $\geq 0.70 \Rightarrow$ allocate 5%" forcing function keyed on evidence that was not in the prompt.
- Nothing measured staleness. Turnover was healthy the entire time the name set was frozen. `cycles_since_name_change` now exists precisely because turnover is not a staleness metric.
- The PM was re-underwriting the wrong book — reading the live account's positions as its own and "exiting" clusters it never held.

All six are fixed. The lesson worth keeping is the shape of the failure: every one was a missing input or a missing measurement, not a bad decision. The model reasoned well over what it was given.

Still open: on the first cycle after the fix, the PM opened NVDA, which was not on the ladder. `opened_off_ladder` now records this. If it recurs, the charter is being skimmed and needs a harder constraint.

Q3. General code architecture health

Good. Protocol-based seams (`DataSource`, `Broker`, `Strategy`) with real alternate implementations, so the simulator is not a mock. Frozen pydantic models. Timezone-aware datetimes enforced at construction. Dependency direction is clean and tested — a spawned subprocess asserts the copilot never imports execution.

880 tests across 86 files, fast and hermetic, with `slow/live` markers keeping the default suite network-free.

Concerns:

1. 270 broad **except** **Exception** handlers, 19 of them `except: pass`. Mostly deliberate — a context builder should degrade rather than kill a cycle — but it is also how the candidate ladder returned empty against a full cache without anyone noticing. Every silent degradation in a decision path should log at warning and surface in the digest. Several now do; not all.
 2. `runner/` is 4,067 lines and holds scheduling, cycle orchestration, order state and kill switches. The natural seam is scheduling vs. cycle execution.
 3. `make_typecheck` fails by design — ~223 mypy strict errors across 46 files, deliberately not gated in CI. Fine as a decision, but it means type errors in new code are invisible unless diffed against a baseline.
 4. **CLUSTERS** covers ~60 tickers. Everything else is uncapped at cluster level. With a 1,600-name universe the concentration cap has large blind spots.
-

Q4. Is the system really working?

Honestly: the trading loop works; the agent loop is unproven.

- The paper momentum strategy has a walk-forward-validated config (`docs/winning_config.md`) and a live equity curve.
- The agent PM sim is +6.4% since inception vs SPY +4.0% over a short window that includes a month of the fixation bug. Encouraging, not evidence. Per this repo's own rule — backtest → walk-forward OOS → ≥30 days paper → live sized down — the PM has not begun that path.
- Agent calibration exists but needs many more graded predictions before the per-agent weights mean anything.

Before live, the three things I would want true and are not yet:

1. `add_episode` wired, so lessons are earned from outcomes.
 2. The candidate ladder proven fresh — the parquet cache has no scheduled refresh; it updates only as a side effect of the trading cycle, whose refresh loop falls back to disk on timeout. Ladders can silently rank week-old momentum. `age_days` and a staleness warning now exist; a refresh job does not.
 3. `trader-mirror` healthy. It has been "Up 3 weeks (unhealthy)" on a pinned image SHA excluded from every rebuild, which makes the live account panel potentially stale.
-

Q5. Are the PM prompts and role definitions clear?

Much better than 24 hours ago, but two structural weaknesses remain.

Fixed: exemplar tickers removed from the PM charter (and from the scout's, which had the same bug); **WHOSE BOOK** block added; anti-inertia scoped to `sim_portfolio.holdings` name by name; the rationale must now account for every name opened or closed.

Weakness A — the charter is ~2,000 words of stacked mandatory rules. HARD RULES, WHOSE BOOK, ANTI-INERTIA, CANDIDATE LADDER, SENTINEL, CREATIVE SCOUT, OPERATOR MANDATES, OPERATOR HOLDS, STOCK PREFERENCE. Six say "mandatory". When everything is mandatory, precedence is undefined — and the model will silently choose. Worth an explicit precedence order.

Weakness B — rules the code does not enforce are suggestions. MAX ETF POSITIONS was charter-only for months and a six-ETF book satisfied every cap that was actually code. Now enforced. The remaining unenforced ones: the sentinel's 70% deployment cap (the PM ran 74% while stating it was respecting 70%), and the 5% creative-scout allocation floor. Either enforce them or stop calling them mandatory.

Q6. Things worth flagging that were not asked

Finding 4 — the mandate detector is vocabulary-locked, and it silently dropped a major strategic instruction. On 2026-08-06 Yan sent a detailed thematic allocation directive ("I want you to... dedicate at least 40% ... to physical AI and quantum... rest of capital 30% to energy infrastructure..."). It was not captured. Reproduced exactly:

```
looks_like_mandate: False      grade_strength: strong
```

Cause: `_clause_is_mandate` requires an instruction verb from a fixed list — `include|add|allocate|buy|hold|own|overweight|underweight|avoid|drop|exclude|look at|keep an eye|consider`. He wrote "dedicate" and "focus". Neither is in the list, so the clause failed the gate even though the strength grader read it as strong. The most natural way to express portfolio direction — "dedicate 40% to X", "put 30% in Y", "keep 20% cash" — has close to zero chance of being caught.

The copilot then refused rather than routing, telling him it was read-only and could not do operational work. That conflates "I cannot place orders" with "I cannot record your instruction". The correct behaviour was to capture it.

Finding 5 — the system cannot express a thematic mandate at all. Even captured, "40% to physical AI and quantum, 30% to energy infrastructure and fintech and air defence" is not executable:

- There is no theme → ticker mapping anywhere. The universe is index constituents plus 21 ETFs.
- `MAX_WEIGHT_PER_STOCK = 0.10`, so a 40% theme needs ≥4 names.
- The candidate ladder ranks by momentum only — no thematic filter, so a theme that is early and not yet trending is invisible to it.
- "Continuously research and be aware of market progress" has no mechanism. The nearest precedent is the hardcoded quantum directive in the scout charter, which is a charter string, not a capability.

This is a feature gap, not a bug. It needs a `config/themes.yaml` mapping themes to tickers, a thematic sleeve in the PM's weight construction, and a scout brief that tracks named themes across weeks.

Silent-failure sweep, 2026-08-06

Everything below ran without erroring, without alerting, and without working. Ordered by how long it had been wrong.

S1 — the entire scorecard had never run (fixed)

`runner._run_memory_grader_async` compared a tz-aware cache index against a tz-naive datetime:

```
hist = s[s.index <= ts0.replace(tzinfo=None)]
```

The parquet index is `datetime64[ms, UTC]`, so pandas raised `TypeError: Invalid comparison` on the first prediction of every pass. The loop sat inside one broad `except Exception`, which logged once a night and moved on.

Consequences, all invisible:

- No prediction was ever graded. `calibration()` returned empty forever.
- Every charter line about weighting agents by track record — the founding premise, “trust calibration over stated confidence” — was reasoning over a table with no rows.
- `source_trust` is updated by `grade_prediction`, so the trust ledger never moved either.
- The exception escaped before `_grade_shadow`, so the counterfactual ledger built on 2026-07-30 never graded a single leg, and the daily memory journal was never written.

No test covered the grader, which is why it could run broken indefinitely. Now: `close_at()` / `covers()` helpers with 11 tests, per-prediction isolation so one bad symbol cannot kill the batch, and grading at the due date rather than the newest bar — the old code scored a 14-day call over however many days happened to have passed.

S2 — the shadow ledger, failing even quieter (fixed)

Same tz bug inside `_grade_shadow.forward_return`, but there the local `except` turned the exception into `None` — and `None` is indistinguishable from “not matured yet”. So /edge looked patiently empty rather than broken.

S3 — the historian was scheduled inside an unrelated `if` (fixed)

```
if _guards_enabled():
    self._scheduler.add_job(..., id="guards")
    self._scheduler.add_job(..., id="historian")    # ← indentation accident
```

`GUARDS_ENABLED` defaults to **false**. Any deployment that had not explicitly opted into position guards never distilled a single lesson, and nothing anywhere would have said so. The weekly learning loop was coupled to a stop-loss feature flag by whitespace.

S4 — the PM was asked to do date arithmetic (fixed)

The SENTINEL RULE says “fired in the last 24 hours” and the prompt handed the model a bare ISO timestamp to compare against a today it had to infer. Date arithmetic is what LLMs are worst at, and the consequence is a 30% cash floor applied or skipped on a misread. Now computed in Python and passed as `fired_within_24h`.

S5 — snapshot age compared two different clocks (fixed)

`datetime.now()` (naive local) minus a UTC snapshot timestamp, inside the reconciliation alert that asks a human whether broker and internal state have diverged. On a CEST machine it misreported the age by two hours.

S6 — **fills_with_symbols** emits **qty**, not **quantity**

Not a pre-existing bug — one I introduced writing the episode recorder, caught because the integration test read the real ledger instead of my fixture. Worth recording as the pattern: my unit tests passed while the code was wrong, because the fixtures agreed with the code rather than with the source.

S7 — the historian has been learning from opinions, not results

A chained consequence of S1 and S2, and the most damaging one.

HISTORIAN_KINDS gives its largest budget — 60 rows — to journal kind prediction_graded, commented "the only rows carrying measured truth". That kind is written by grade_prediction, which never ran. And measured_edge, which the charter says "is measurement, not opinion — it outranks anything in the prose", is built from mem.edge_report() over the shadow ledger, which never graded a leg.

So every weekly distillation has seen committee prose and portfolio changes, and zero measured outcomes. The historian was told to prefer measurement over narrative and then handed only narrative. Its lessons are, structurally, summaries of what the desk said rather than what the desk achieved.

Both inputs unblock automatically now that S1 and S2 are fixed — no further code needed, but the lesson book written before today should be read as unevidenced.

S8 — the index universe is 8 weeks stale, and cannot self-refresh

config/universes.generated.yaml was last written 12 June. scripts/refresh_universes.py exists and is scheduled nowhere — not in the runner, not in ofelia, not in cron. So sp500, nasdaq100 and russell1000 are two months of index changes out of date: names removed from the index are still rankable, names added are invisible to the candidate ladder.

Worse, it cannot simply be scheduled: config/ is bind-mounted read-only into the container, so nothing running on the VPS can write that file. Fixing it properly means either an env-overridable generated-universes path pointing at state/, or a host-side cron that runs the script and commits. That is a design decision, not a patch.

The common thread in S1–S3: a broad except Exception around a loop that is supposed to do work, with no assertion anywhere that the work happened. The system reports what it attempted, not what it achieved. Every counter that matters — predictions graded, episodes recorded, legs filled — should be visible, and a zero that persists should alert.

Priority order

Done 2026-08-06: add_episode wired (memory/episodes.py, nightly, idempotent, reconstructed from the fill ledger); mandate vocabulary widened plus percentage-allocation detection; copilot charter rule 11b — record direction, never refuse it; scheduled price_cache_refresh weekdays 21:40 UTC that alerts if the cache is still stale afterwards; plus S1–S5 above.

Still open:

	Item	Effort
1	Consume cited_lessons; retire lessons nobody cites	S
2	Explicit precedence order in the PM charter (six blocks say "mandatory")	S
3	Enforce the sentinel deployment cap in code, not prose	S
4	Fix trader-mirror (unhealthy 3 weeks, pinned SHA, excluded from rebuilds)	S
5	Assert the work happened telemetry done — ops_watch.check_learning_loops + check_recent_errors	–
5b	Index universe refresh: pick a mechanism (read-only config/ blocks the obvious one)	M
6	Point historian lesson evidence at real episode IDs now that they exist	M
7	Thematic mandate capability (config/themes.yaml + a PM sleeve)	L
8	Extend CLUSTERS, or derive clusters from correlation	M
9	Split runner/ along the scheduling / execution seam	M

Nothing above is a reason not to keep running paper. Item 5 is the one I would not skip: the whole point of S1–S3 is that this system could not tell the difference between working and not.